APS Failure Detection in Scania Trucks

A Comparative Study of Machine Learning Classifiers

Marcus Giannini | CS 460 Intro. to Applied Machine Learning | May 2026

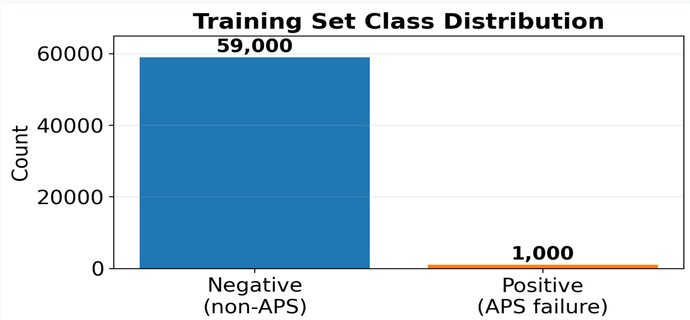
Introduction / Objective

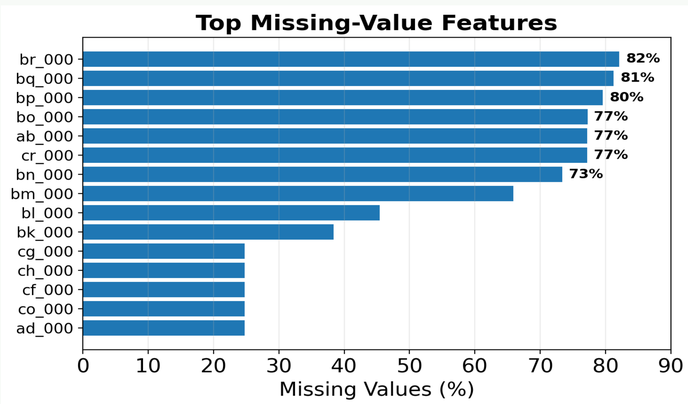
This project predicts whether a heavy Scania truck failure is caused by the Air Pressure System (APS) or by another component, using anonymized sensor data from trucks in everyday operation. The dataset is severely imbalanced and comes with an official cost metric where missing a real APS failure is 50 times more expensive than flagging a healthy truck. The objective was to train, tune, and compare six machine learning classifiers, then evaluate them on multiple metrics to determine which model is genuinely best for this kind of industrial maintenance problem.

Dataset

- Scania APS Failure dataset
- 60,000 training rows / 16,000 test rows
- 170 anonymized numerical features + binary class label
- Training class balance: 59,000 negative / 1,000 positive
- Official cost metric: 10×FP + 500×FN

Exploratory Data Analysis





- Severe class imbalance made recall and F1 more useful than accuracy alone.
- Several columns had very high missingness, so those columns were dropped before imputation.
- Many features were skewed and had outliers, so median imputation and scaling were used.

Methodology

Drop high missing cols — Median imputation — Standard scaling — GridSearchCV 5-fold CV — Held-out test set

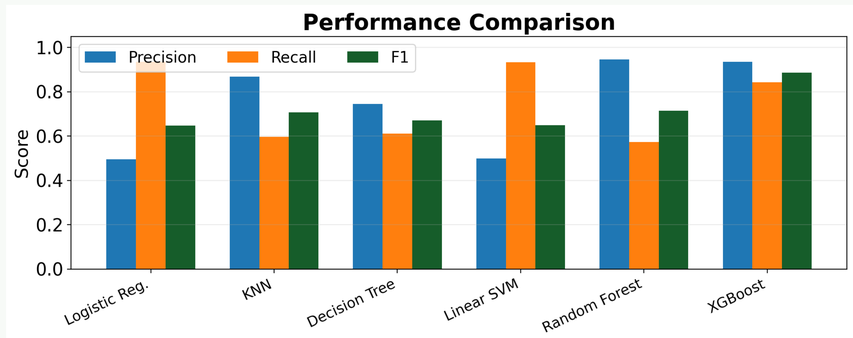
- Preprocessing was fit on the training data only, then applied to the test data.
- Six tuned models were compared: Logistic Regression, KNN, Decision Tree, Linear SVM, Random Forest, and XGBoost.
- GridSearchCV used F1 as the tuning score because the positive class is rare.

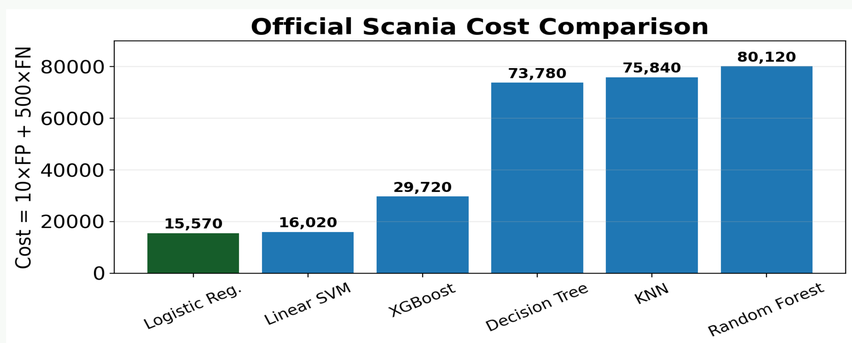
Metrics Used
Accuracy • Precision • Recall • F1
ROC-AUC • Confusion matrix • Cost

Official Scania Cost Metric
Cost = 10 × FP + 500 × FN
False negatives are 50× more expensive.

Results

Model	Acc.	Prec.	Recall	F1	AUC	FN	FP	Cost
Logistic Reg.	**0.976**	**0.496**	**0.936**	**0.648**	**0.989**	**24**	**357**	**15,570**
KNN	0.988	0.868	0.597	0.708	0.886	151	34	75,840
Decision Tree	0.986	0.746	0.611	0.672	0.803	146	78	73,780
Linear SVM	0.976	0.499	0.933	0.650	0.983	25	352	16,020
Random Forest	0.989	0.947	0.573	0.714	0.993	160	12	80,120
XGBoost	**0.995**	**0.935**	**0.843**	**0.886**	**0.995**	**59**	**22**	**29,720**





Main interpretation
At first, XGBoost looks like the best answer because it has the strongest balanced performance. But once the official cost metric is applied, Logistic Regression becomes the better operational choice because it misses fewer true APS failures.

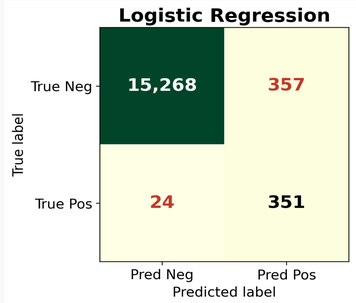
Key Finding

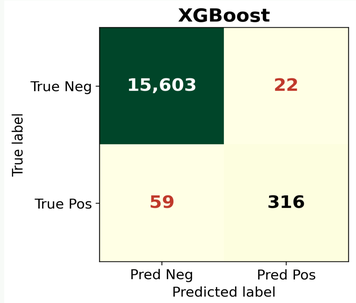
The "best" model depends on the metric.

XGBoost wins by F1 / accuracy / ROC-AUC.
Logistic Regression wins by official Scania cost.
Random Forest wins by precision.
Logistic Regression wins by recall.

Logistic Regression missed 24 failures. XGBoost missed 59. Since each missed failure costs 500, fewer false negatives mattered more.

Cost-Sensitive Comparison





Logistic Regression
FN = 24 | Cost = 15,570

XGBoost
FN = 59 | Cost = 29,720

Conclusions

- XGBoost was the strongest balanced classifier (F1 = 0.886, AUC = 0.995).
- Logistic Regression was the lowest-cost model under the official Scania metric.
- The default 0.5 threshold is probably not ideal for this cost-sensitive problem.
- For a maintenance setting, reducing false negatives should matter more than maximizing accuracy.
- Class weighting was effective for linear models (LR, SVM) but did not help tree-based models recover the recall they lost on the imbalanced data.

Future Work

- Tune the classification threshold directly against the Scania cost metric.
- Compare class weighting strategies more systematically.
- Try PCA or feature selection to reduce the feature space.
- Test cost-sensitive evaluation with different cost ratios to see how robust the rankings are.

Tools & Code

- Implemented in Python using pandas, NumPy, scikit-learn, matplotlib, and XGBoost.
- The official test set was held out until final evaluation. random_state=42 was used where possible.
- Hyperparameter tuning used GridSearchCV with 5-fold cross-validation. The grid for XGBoost included n_estimators, max_depth, and learning_rate.
- Class imbalance was handled with class_weight="balanced" for most models, and scale_pos_weight for XGBoost.

References: He & Garcia (2009), Learning from Imbalanced Data, IEEE TKDE 21(9). | Lindgren & Biteus (2016), APS Failure at Scania Trucks Data Set, UCI ML Repository. | Müller & Guido (2016), Introduction to Machine Learning with Python, O'Reilly. | Scania CV AB (2016), Dataset description and IDA 2016 Industrial Challenge cost metric. | scikit-learn.org | xgboost.readthedocs.io